MUST READ FOR CLEAR UNDERSTANDING

Imagine you are asking an AI to give answers in a proper format, not random text. This is called **Structured Output** in LangChain.

Think like this: you ask the AI to give information in boxes 📦 instead of messy sentences.

Example output we want:

Name: Rahul
Age: 20
City: Delhi

Now in LangChain there are 3 ways to define this structure:
JSON, TypedDict, and Pydantic. Let's explain them like you are teaching a small child 👶.

1️⃣ JSON – The Simple Box 📦

JSON is just a **simple dictionary format**.

Think of it like a **labelled box** where you write key and value.

Example:

JSON

```
{
 "name": "Rahul",
 "age": 20,
 "city": "Delhi"
}
```

Child example

Teacher: "Write student details."

Student writes:

```
Name → Rahul
Age → 20
City → Delhi
```

That's JSON.

In LangChain

Python Run

```python
{
 "name": str,
 "age": int,
 "city": str
}
```

Problem

❌ It **does not check strongly** if data is correct.

Example:

```
age: "twenty"
```

JSON may still accept it.

So it is **simple but not very safe**.

2️⃣ TypedDict – A Slightly Strict Teacher 📚

TypedDict comes from Python typing.

It tells Python:

"These fields must exist and must have these types."

Example:

```
from typing import TypedDict

class Student(TypedDict):
    name: str
    age: int
    city: str
```

Now Python expects:

name → text
age → number
city → text

Child example

Teacher gives a form:

Name: _____
Age: _____

City: _____

You must fill the correct type.

Age must be a number.

Advantage

✔ clearer than JSON
✔ Python understands the types

Problem

❌ Not very strict at runtime
❌ No validation rules

3 Pydantic – The Smart Teacher 🧠

Pydantic is the **most powerful**.

It **checks data, validates it, and gives errors**.

Example:

```
from pydantic import BaseModel

class Student(BaseModel):

    name: str

    age: int

    city: str
```

Now if AI returns:

age = "twenty"

Pydantic will say ❌ **Error! Age must be a number**

Child example

Teacher checks the form carefully:

Name: Rahul

Age: twenty

Teacher says:

❌ Wrong
Age must be a number.

Advantage

✔ Very strict
✔ Validates data
✔ Gives errors
✔ Best for APIs and production

****

Interview Ready

1. JSON

Schema

- JSON defines the **structure of the output using key–value pairs**.
- It tells the LLM what fields should exist.

Example schema idea:

JSON

```
{
  "name": "string",
  "age": "integer",
  "city": "string"
}
```

Validation

- JSON itself **does not perform validation**.
- If the LLM outputs incorrect types, the program must check it manually.

Example issue:

```
age: "twenty"
```

JSON will still accept it unless additional validation is written.

✅ **Summary:**

JSON provides **schema only**, but **no built-in validation**.

2. TypedDict

Schema

TypedDict defines the schema using **Python type annotations**.

Example:

Python Run

```python
from typing import TypedDict

class Student(TypedDict):
    name: str
    age: int
    city: str
```

This schema clearly specifies:

- name → string
- age → integer
- city → string

Validation

- Provides **static type checking** (mainly during development).
- Does **not strongly validate data at runtime**.

If wrong data comes from the LLM, Python may still accept it unless additional checks are added.

✅ **Summary:**

TypedDict provides **clear schema with type hints**, but **limited runtime validation**.

3. Pydantic

Pydantic defines a schema using **Python models with built-in validation rules.**

Example:

```python
from pydantic import BaseModel

class Student(BaseModel):
    name: str
    age: int
    city: str
```

Schema

- Clearly defines fields and data types.
- Supports complex schemas like nested objects and constraints.

Validation

- Performs **automatic runtime validation.**

- Raises errors if:
  - data types are incorrect
  - required fields are missing
  - constraints are violated

Example:

```
age = "twenty"
```

Pydantic will raise a **validation error.**

✅ **Summary:**

Pydantic provides **both schema definition and strong runtime validation.**

QUICK COMPARSION

Method	Schema	Validation
JSON	Basic structure definition	No built-in validation
TypedDict	Schema with Python type hints	Limited/static validation
Pydantic	Full schema definition	Strong runtime validation